

Deep Learning-Assisted Improved Differential Fault Attacks on Lightweight Stream Ciphers

Kok Ping Lim, Dongyang Jia, Iftekhhar Salam

School of Computing and Data Science,
Xiamen University Malaysia, Sepang 43900, Malaysia
{CYS2209667, CME2209122, iftekhar.salam}@xmu.edu.my

Abstract. Lightweight cryptographic primitives are widely deployed in resource-constraint environment, particularly in the Internet of Things (IoT) devices. Due to their public accessibility, these devices are vulnerable to physical attacks, especially fault attacks. Recently, deep learning-based cryptanalytic techniques have demonstrated promising results; however, their application to fault attacks remains limited, particularly for stream ciphers. In this work, we investigate the feasibility of deep learning assisted differential fault attack on three lightweight stream ciphers, namely ACORNv3, MORUSv2 and ATOM, under a relaxed fault model, where a single-bit bit-flipping fault is injected at an unknown location. We train multilayer perceptron (MLP) models to identify the fault locations. Experimental results show that the trained models achieve high identification accuracies of 0.999880, 0.999231 and 0.823568 for ACORNv3, MORUSv2 and ATOM, respectively, and outperform traditional signature-based methods. For the secret recovery process, we introduce a threshold-based method to optimize the number of fault injections required to recover the secret information. The results show that the initial state of ACORN can be recovered with 21 to 34 faults; while MORUS requires 213 to 248 faults, with at most 6 bits of guessing. Both attacks reduce the attack complexity compared to existing works. For ATOM, the results show that it possesses a higher security margin, as majority of state bits in the Non-linear Feedback Shift Register (NFSR) can only be recovered under a precise control model. To the best of our knowledge, this work provides the first experimental results of differential fault attacks on ATOM.

Keywords: Differential Fault Attack · MLP · ACORN · MORUS · ATOM

1 Introduction

Over the past two decades, various cryptographic competitions, such as eSTREAM, the Competition for Authenticated Encryption: Security, Applicability, and Robustness (CAESAR) and National Institute of Standards and Technology (NIST) Lightweight Cryptography Project were called for the design and third-party analysis of lightweight cryptography primitives. Lightweight cryptography is designed to ensure secure communication for resource-constrained

devices, e.g., Internet of Things (IoT) devices, as traditional ciphers are intended to operate in resource-rich environments. Many cryptanalysis techniques, such as cube attack, side channel attack and fault attack are applied to the lightweight ciphers. Among these attacks, fault attacks have gained significant attention in recent years due to their effectiveness against physical hardware and the ongoing advancement of fault injection techniques. Since IoT devices are commonly deployed in publicly accessible areas, adversaries often have physical access to them, making these devices viable targets for fault attacks [23].

Fault attack was first introduced by Boneh et al. [4] and targets physical hardware rather than the cryptographic algorithm itself. They applied the attack to RSA and Robin signature schemes by introducing hardware faults to factor the large prime modulus N . In the same year, Biham and Shamir [3] implemented a variant of fault attack, differential fault attack (DFA), and applied it against the Data Encryption Standard (DES) to recover the 56-bit secret key. The attack works by inducing an error (fault) during the operation of the cipher to obtain the faulty outputs. By analysing the differences between fault-free and faulty outputs, some secret information of the cipher, such as the internal state or secret key can be recovered. Hoch and Shamir [8] further extended DFA against RC4, LILI-128 and SOBER-t32, introducing DFA on stream ciphers.

Following these foundational works, differential fault attacks were further extended and refined, and subsequently applied to evaluate the security of a wide range of cryptographic primitives. Orumiehchiha et al. [15] applied the first DFA against WG-8 stream cipher to recover the initial state and secret key of the cipher using 6 faults with the time complexity of 2^{24} . Ma, Tian and Qi [11] applied DFA against LIZARD stream cipher. Using 6 faults, the initial state of LIZARD can be recovered. Salam et al. [20] performed bit-flipping, probabilistic random and deterministic random fault attacks on Grain128AEAD to recover the initial state. Salam et al. [21] managed to recover the initial state and secret key of CLX-128 by applying bit-flipping and random fault attack, with 54 and 134 faults, respectively. Rostami et al. [18] applied a random fault attack against Enocoro-80 and Enocoro-128v2. The result shows that the attack only required 2 faults to recover the internal state of Enocoro-80 and Enocoro-128v2 with the time complexity of 2^{16} and 2^{29} , respectively. Radheshwar and Roy [17] applied DFA against ChaosForge stream cipher and managed to recover the internal state and the secret key. Prajasantosa and Salam [16] were able to recover 58 state bits and 15 key bits of TinyJAMBU with bit-flipping and random fault attacks, under different control models.

Recently, some works utilized deep learning methods to assist cryptanalysis on various lightweight block ciphers, such as finding the distinguishers of block ciphers [7,9,27]. The deep learning approach has also been used to predict the secret key of lightweight block ciphers, such as S-DES and S-AES [10]. In terms of deep learning assisted fault attack, Baksi et al. [1] developed multilayer perceptron (MLP) models to identify the fault location for lightweight stream ciphers, Grain128a and Lizard, which achieves better performance than the traditional signature-based method. Cheng et al. [5] proposed a deep learning-based fault

analysis, using MLP and Convolution Neural Network (CNN) to recover the key of AES with 1 fault and an average of 1488 ciphertexts. Saha et al. [19] implemented a fault attack leakage assessment test with deep learning to detect fault-induced leakage in the ciphertext distributions. Mondal et al. [14] developed a deep neural network for identifying the fault location of Subterranean 2.0 and recovered the embedded secret key with no more than 5 faults.

The existing literature shows that differential fault attacks remain a popular cryptanalytic technique for state recovery or key recovery. In addition, the integration of deep learning in fault attack provides a new direction in enhancing the performance of the attack. Given the limited research on deep learning-assisted DFA, the feasibility of such fault attacks remains largely underexplored. To address this, we propose a deep learning-assisted DFA framework for enhancing differential fault attacks and apply it to ACORNv3, MORUSv2, and ATOM.

1.1 Existing Differential Fault Attacks on ACORN and MORUS

Dalai and Roy [6] implemented differential fault attacks on the first and second version of ACORN. By precisely injecting 326 faults and guessing 120 state bits, the full internal state of cipher at the 39-th clocking of encryption phase can be recovered, with an attack complexity of 2^{120} . Zhang et al. [28] applied DFA against ACORNv2 under the fault model of no control over the fault location. By analyzing the occurrence of 1 in the differential keystream, the fault location were identified with a high probability of 97.08%. For equations solving, they used linearization and guess-and-determine strategies to solve the equations and recover the initial state. The linearization strategy required 31 to 88 faults to recover the initial state with an attack complexity of $c \cdot 2^{179.19-1.76n}$, where n was the number of fault injection, and c was the time complexity of solving the equations. For the guess-and-determine method, guessing 67 state bits could recover the initial state of the cipher with 41 fault experiments.

Siddhanti et al. [22] extended DFA to ACORNv3, optimizing the classical differential framework by incorporating both forward and backward state-update equations and fixing selected bits before SAT solving. In their attack, the fault locations were identified using the signature-based method. Due to the large state size of ACORN, the attack was optimized by guessing 20 state bits. Their work showed that the initial state of ACORNv3 can be recovered using 9 faults and 1200-bit keystream, with the time complexity of $2^{25.40}$. Zhang et al. [29] also performed DFA on ACORNv3, following the same attack process as in [28]. To identify the fault location, they introduced the “unique” versus “non-unique” differential sets method. By incorporating the keystream extension and high probability priority strategies, the accuracy of their proposed method was 99.998%, using 163-bit differential keystream. For the equations solving, a guess-and-determine method was used to recover the full initial state of the cipher. The attack complexity is $c \cdot 2^{146.5-3.52n}$, where n is the number of faults such that $26 < n < 43$ faults and where c was the time complexity of solving the equations.

In short, most of the existing works of DFA against ACORN requires the guess-and-determine method to recover the state. In addition, some of the works focus on retrieving only linear equations, that require more fault injections to recover the state of the cipher. Additionally, the attacks based on fault model with no control over the fault location rely on traditional signature-based method to identify fault location. Hence, this presents a gap for investigating the use of deep learning for identifying the fault locations and improving the attack complexity. Furthermore, investigation can be conducted by incorporating high degree equations with a threshold-based technique to optimize the number of faults.

Wong et al. [24] performed a theoretical analysis of DFA on MORUS, targeting the bit-wise AND operation of the keystream generation function, where a random fault is injected into one of the two involved registers. With these faults, an adversary can reveal the non-zero state bits in the targeted register whenever the differential keystream bits are 1. Repeated fault injections allow these non-zero bits to be recovered with high probability, while the remaining bits are assumed to be 0 with high confidence. However, the attack only achieved partial state recovery and its reliance on precise control over the fault location is a relatively strong assumption; leaving open the possibility of achieving full state recovery of MORUS with a weaker assumption.

1.2 Our Contributions

We propose a deep learning assisted differential fault attack framework under the single-bit bit-flipping fault model with no control over the fault location. The contribution of this work is summarized as follows:

MLP Models for Fault Location Identification. We generate differential keystream data sets for ACORNv3, MORUSv2 and ATOM, and design and train MLP models to identify the fault locations for these ciphers. The models achieve accuracies of 0.999880, 0.999231 and 0.823568 for ACORNv3, MORUSv2 and ATOM, respectively; significantly improving over the traditional signature-based method. Table 1 summarizes the results of fault location identification in this work.

Table 1. Summary of fault location identification accuracy across different ciphers

Cipher	Method	Keystream length	Accuracy	Ref.
ACORNv3	MLP (this work)	152 bits	0.999880	Sec. 4.1
	Set-based	149 bits	0.999774	Ref. [29]
	Signature-based	152 bits	0.999747	Sec. 4.1
MORUSv2	MLP (this work)	384 bits	0.999231	Sec. 4.2
	Signature-based	384 bits	0.952606	Sec. 4.2
ATOM	MLP (this work)	56 bits	0.823568	Sec. 4.3
	Signature-based	56 bits	0.588976	Sec. 4.3

Threshold-based Attack Framework. We introduce a threshold-based framework to minimize the number of fault injections. Meanwhile, we include all equations into the equation system for secret recovery, regardless of the degree. We achieve a full state recovery on ACORNv3 and MORUSv2, improving the existing results in terms of attack complexity and number of recovered bits, respectively. We also implemented the first DFA against ATOM and the result shows that ATOM possesses a higher security margin compared to ACORNv3 and MORUSv2. Table 2 summarizes our results of DFA.

Table 2. Summary of differential fault attacks on ACORNv3, MORUSv2, and ATOM

Cipher	Fault type	# Faults	# Recovered bits	Complexity	Ref.
ACORNv3	Bit-flipping	9	Full initial state	$2^{25.40}$	Ref. [22]
	Bit-flipping	27–42	Full initial state	$c \cdot 2^{146.5-3.52n}$	Ref. [29]
	Bit-flipping	21–34	Full initial state	Negligible	Sec. 4.1
MORUSv2	Random	–	256 state bits	–	Ref. [24]
	Bit-flipping	213–248	Full initial state	2^6 (At most)	Sec. 4.2
ATOM	Bit-flipping	46	Majority of NFSR state	$\geq 2^{60}$	Sec. 4.3

Organization of Paper. We provide a brief background on fault location identification and overview of ACORNv3, MORUSv2 and ATOM in Section 2. In Section 3, the attack procedure, including the implementation of MLP model and the proposed threshold-based method are discussed. The implementation of attacks and results for the three ciphers are discussed in Section 4. Finally, we conclude the paper in Section 5. All implementation details, including source code and dataset, can be accessed through the GitHub repository ¹.

2 Preliminaries and Background

2.1 Fault Location Identification

Our proposed deep learning assisted fault attack framework works under the fault model of no-control over the fault location. In this model, fault location identification is a critical process, as it can affect the secret recovery process. Consider a toy cipher with 4 bits state, $s_0||s_1||s_2||s_3 = 0||0||0||1$, where the first keystream bit is defined as:

$$z_0 = s_0 \oplus s_1 \oplus s_1 s_2 \oplus s_2 s_3. \quad (1)$$

During the attack, a fault is injected at the location s_2 , and thus the value of s_2 is complement. The faulty keystream equation can be rewritten as:

$$z'_0 = s_0 \oplus s_1 \oplus s_1 s'_2 \oplus s'_2 s_3, \quad (2)$$

¹ https://github.com/kokping0605/Deep_Learning_Assisted_Fault_Attack_Framework.git

where $s'_2 = s_2 \oplus 1$. By XOR-ing Equation 1 and Equation 2, we can obtain the differential:

$$\begin{aligned}
z_0 \oplus z'_0 &= (s_0 \oplus s_1 \oplus s_1 s_2 \oplus s_2 s_3) \oplus (s_0 \oplus s_1 \oplus s_1 s'_2 \oplus s'_2 s_3) \\
&= s_1 s_2 \oplus s_2 s_3 \oplus s_1 s'_2 \oplus s'_2 s_3 \\
&= s_1(s_2 \oplus s'_2) \oplus s_3(s_2 \oplus s'_2) \\
&= s_1 \oplus s_3.
\end{aligned} \tag{3}$$

Substituting the values into the equations, we can find that Equation 3 is valid. Note that in the no control model, the adversary has no knowledge about the fault location. Suppose the fault is injected at s_2 , but the adversary misidentifies the fault location as s_3 , resulting in the following differential equation instead:

$$\begin{aligned}
z_0 \oplus z'_0 &= (s_0 \oplus s_1 \oplus s_1 s_2 \oplus s_2 s_3) \oplus (s_0 \oplus s_1 \oplus s_1 s_2 \oplus s_2 s'_3) \\
&= s_2 s_3 \oplus s_2 s'_3 \\
&= s_2(s_3 \oplus s'_3) \\
&= s_2.
\end{aligned} \tag{4}$$

Substituting the values into the equation, it shows that the wrong value is revealed for s_2 due to the misidentification. Thus, it is important to identify the correct fault location, under the no control model, to ensure that the valid differential can be derived. There are a number of methods to identify the fault location. Signature-based method is one of the most popular methods to identify the fault location. One can refer to [12,13,22] for the details of signature-based method.

2.2 Specification of Ciphers

ACORNv3 is one of the algorithms in the final portfolio of the CAESAR competition for the use case of lightweight application, while MORUSv2 is one of the additional finalists for high performance application in the same competition. ATOM, on the other hand, is a stream cipher proposed by Banik et al. [2], that utilized a double key filter. In our fault attacks, we mainly focus on the encryption phase of the ciphers, and thus only the encryption process of each cipher is described in the following sections. One can refer to [25,26,2] for the detailed specification of each cipher.

ACORNv3 Let $S = \{s_0, s_1, \dots, s_{292}\}$ denote the initial state of ACORNv3. In the encryption phase, the cipher processes the plaintext $P = \{p_0, p_1, \dots, p_{mlen-1}\}$ and output the ciphertext $C = \{c_0, c_1, \dots, c_{mlen-1}\}$. The cipher utilizes the output function $ks(S)$ to generate a keystream bit z_i , where $i = 0, 1, \dots, mlen - 1$. The output function is defined as:

$$\begin{aligned}
ks(S) &= s_{12} \oplus s_{154} \oplus s_{61} s_{235} \oplus s_{93} s_{235} \oplus s_{61} s_{93} \\
&\quad \oplus s_{111} s_{230} \oplus s_{66} \overline{s_{230}}.
\end{aligned} \tag{5}$$

Note that $\overline{s_{230}}$ is the complement of s_{230} . Besides, a feedback function $fb(S)$ is used to compute a feedback bit y , where the function is defined as

$$fb(S) = s_0 \oplus \overline{s_{107}} \oplus s_{23}s_{244} \oplus s_{160}s_{244} \oplus s_{23}s_{160} \oplus s_{196}. \quad (6)$$

Note that $\overline{s_{107}}$ is the complement of the state bit s_{107} . The encryption process of ACORNv3 is described in Algorithm 1.

Algorithm 1 Encryption phase of ACORNv3

```

Initialize list  $Z$ 
for  $i = 0$  to  $m_{len} - 1$  do
     $s_{289} = s_{289} \oplus s_{235} \oplus s_{230}$ 
     $s_{230} = s_{230} \oplus s_{196} \oplus s_{193}$ 
     $s_{193} = s_{193} \oplus s_{160} \oplus s_{154}$ 
     $s_{154} = s_{154} \oplus s_{111} \oplus s_{107}$ 
     $s_{107} = s_{107} \oplus s_{66} \oplus s_{61}$ 
     $s_{61} = s_{61} \oplus s_{23} \oplus s_0$ 
     $z_i = ks(S)$ 
    Append  $z_i$  to  $Z$ 
     $y = fb(S)$ 
    for  $j = 0$  to 291 do
         $s_j = s_{j+1}$ 
    end for
     $s_{292} = y \oplus p_i$ 
end for
return  $Z = \{z_0, z_1, \dots, z_{m_{len}-1}\}$ 

```

From Algorithm 1, the state bits s_{61} , s_{107} , s_{154} , s_{193} , s_{230} and s_{289} will undergo an intermediate update before a keystream bit z_i is generated. Once the six state bits have undergone the intermediate update, the state is input to the output function and feedback function to produce the keystream bit and feedback bit, respectively. After a keystream bit is generated, the cipher updates the internal state by shifting, but the last state bit s_{292} is updated by XOR-ing the feedback bit y with the plaintext bit p_i .

MORUSv2 We apply the attack on MORUS-640-128, which is one of the three variants of MORUSv2. It takes a 128-bit secret key, with an internal state of 640 bits. The state is constructed of 5 registers, denoted as $S = S_0 \| S_1 \| S_2 \| S_3 \| S_4$, where each register hold 128-bit data. In each step of the encryption phase, the cipher processes a 128-bit plaintext block. Let the plaintext P consist of y blocks, where the plaintext block is noted as P_i , $i = \{0, 1, \dots, y-1\}$. If the last plaintext block P_{y-1} is not a full block, a 0 valued string is padded to that block. The keystream block Z_i computed by $S_0 \oplus (S_1 \ll 96) \oplus S_2 S_3$, where S_1 is rotated to the left by 96 bits, as defined in Algorithm 2.

Algorithm 2 Encryption phase of MORUSv2

```
Initialize list  $Z$ 
for  $i = 0$  to  $y - 1$  do
     $Z_i = S_0 \oplus (S_1 \ll 96) \oplus S_2 S_3$ 
    Append  $Z_i$  to  $Z$ 
    Update the state with state update function
end for
return  $Z = \{Z_0, Z_1, \dots, Z_{y-1}\}$ 
```

Algorithm 3 Encryption phase of ATOM

```
Initialize list  $Z$ 
for  $i = 0$  to  $m_{len} - 1$  do
     $z_i = O(B, L)$ 
    Append  $z_i$  to  $Z$ 
     $cnt = l_{62} \parallel l_{63} \parallel l_{64} \parallel l_{65} \parallel l_{66} \parallel l_{67} \parallel l_{68}$ 
     $fb\_B = G(B) \oplus l_0 \oplus k_{cnt} \oplus k_{511+i \bmod 128}$ 
    for  $j = 0$  to 88 do
         $b_j = b_{j+1}$ 
    end for
     $b_{89} = fb\_B$ 
     $fb\_L = F(L)$ 
    for  $j = 0$  to 67 do
         $l_j = l_{j+1}$ 
    end for
     $l_{68} = fb\_L$ 
end for
return  $Z = \{z_0, z_1, \dots, z_{m_{len}-1}\}$ 
```

ATOM ATOM consists of a linear feedback shift register (LFSR) and a non-linear feedback shift register (NFSR). Let the initial state of LFSR and NFSR as $L = \{l_0, l_1, \dots, l_{68}\}$ and $B = \{b_0, b_1, \dots, b_{89}\}$, respectively. During encryption, the keystream bit is computed as:

$$\begin{aligned} O(B, L) = & b_1 \oplus b_5 \oplus b_{11} \oplus b_{22} \oplus b_{36} \oplus b_{53} \oplus b_{72} \oplus b_{80} \\ & \oplus b_{84} \oplus l_5 l_{16} \oplus l_{13} l_{15} \oplus l_{30} l_{42} \oplus l_{22} l_{67} \\ & \oplus h(l_7, l_{33}, l_{38}, l_{50}, l_{59}, l_{62}, b_{85}, b_{41}, b_9). \end{aligned} \quad (7)$$

Note that $h(l_7, l_{33}, l_{38}, l_{50}, l_{59}, l_{62}, b_{85}, b_{41}, b_9)$ is a degree 5 function. Both LFSR and NFSR possess the corresponding feedback function to update the state. The feedback function for the LFSR and NFSR are denoted by $F(L)$ and $G(B)$, respectively, and are defined as:

$$F(L) = l_0 \oplus l_5 \oplus l_{12} \oplus l_{22} \oplus l_{28} \oplus l_{37} \oplus l_{45} \oplus l_{58}, \quad (8)$$

$$\begin{aligned} G(B) = & b_0 \oplus b_{24} \oplus b_{49} \oplus b_{79} \oplus b_{84} \oplus b_3 b_{59} \oplus b_{10} b_{12} \\ & \oplus b_{15} b_{16} \oplus b_{25} b_{53} \oplus b_{35} b_{42} \oplus b_{55} b_{58} \oplus b_{60} b_{74} \\ & \oplus b_{20} b_{22} b_{23} \oplus b_{62} b_{68} b_{72} \oplus b_{77} b_{80} b_{81} b_{83}. \end{aligned} \quad (9)$$

Algorithm 3 defines the encryption phase of ATOM. Note that last 7 bits in the LFSR are interpreted as a decimal number cnt to determine which key bit k is involved in the computation of the feedback bit for the NFSR. The NFSR feedback bit is computed by XOR-ing the output of update function with l_0 , and two key bits chosen through the cnt and modulo of current clock. The value of cnt is non-deterministic, as it depends on the current state of the LFSR. The utilization of double key filter ensures that the generated keystream possesses a longer period. Excluding the last state bits in both NFSR and LFSR, the remaining state bits in both registers are updated by shifting.

3 Attack Description

Our attacks are conducted in a simulation-based environment, where the fault injection is simulated by code, rather than implementing the attack on a physical device. The attack assumes a known-plaintext attack (KPA) model, where the adversary has access to specific keystream bits. Besides, the following assumptions are made:

- The adversary can inject a single-bit bit-flipping fault at the initial state
- The adversary has no control over the fault location.
- The adversary can always reset the cipher and run the cipher with the embedded key and initialization vector (IV) to observe the keystream.

The attack consists of two phases, the fault location identification and secret recovery process. The details of these two phases are discussed in the following section.

3.1 Fault Location Identification

The fault location identification is the critical process in our attack, as the fault is injected to unknown location. Without the knowledge of fault location, we cannot obtain the valid differential keystream equations to recover the secret of the cipher. Thus, MLP models are trained to identify the fault location using differential keystream datasets.

Dataset Generation Since there is no public dataset for fault location identification for each cipher, we generate the datasets by ourselves. The procedure of dataset generation for each cipher is defined in Algorithm 4. For each possible fault location f of the cipher, 1536 samples are generated, each with a random key, K and IV, V . The key-IV pair is used to run the cipher for producing the fault-free keystream Z . After that, the cipher is reset and run with the same key-IV pair. At this time, a bit-flipping fault is injected to location f in the initial state of the cipher. The faulty initial state is then used to produce the faulty keystream Z' . The differential keystream is then computed by XOR-ing Z with Z' . Finally, the key-IV pair, fault-free keystream, faulty keystream, differential keystream and the fault location are recorded into the csv file.

Algorithm 4 Dataset Generation

```
for each possible fault location  $f$  of the cipher do
  for  $j = 1$  to 1536 do
    Generate random key  $K$  and IV  $V$ 
    Run the cipher and obtain fault-free keystream  $Z$ 
    Reset the cipher to initial state
    Inject bit-flipping fault at location  $f$ 
    Run the cipher and obtain faulty keystream  $Z'$ 
    Obtain differential keystream  $\Delta Z = Z \oplus Z'$ 
    if  $i \leq 1024$  then
      Write  $K, V, Z, Z', \Delta Z, f$  to training.csv
    else if  $i \leq 1280$  then
      Write  $K, V, Z, Z', \Delta Z, f$  to testing.csv
    else
      Write  $K, V, Z, Z', \Delta Z, f$  to validation.csv
    end if
  end for
end for
```

For each cipher, three datasets are generated, one for training, one for evaluation and one for validation. To enhance the neural network model in capturing the pattern of fault propagation, the train-test ratio is set as 80:20. The size of training dataset is set as $n \cdot 2^{10}$, while testing and validation datasets are $n \cdot 2^8$, where n is the number of possible fault locations. Hence, each fault location has 2^{10} samples for training, 2^8 samples for testing. Note that the validation dataset is used to evaluate the performance of the model during the training. The length of the differential keystream is set to the range between 50% and 60% of the state size of the cipher, as referred to the work of Baksi et al [1].

Implementation of Multilayer Perceptron Model Using the generated datasets, we employ multilayer perceptron (MLP) models to identify the fault locations. The implementation of the models are done with the Keras library provided in the TensorFlow framework, using the Google Colab. The number of neurons in the input layer is the length of the differential keystream, while the number of neurons in the output layer is the size of the internal state of the cipher, that is the number of possible fault location. For the number of hidden layers, number of neurons or other hyperparameters, an exhaustive search is used to optimize the parameters.

The model is compiled with the Adam optimizer and the sparse categorical cross entropy loss function. Since this is a classification problem, accuracy is chosen as the metric. The model is trained for a maximum of 50 epochs and the early-stopping criterion based on validation accuracy is applied. If there is no improvement in validation accuracy for seven consecutive epochs, the training process is stopped, and the weights of the best validation accuracy are restored. The hyperparameters including the number of neurons and hidden layers, dropout

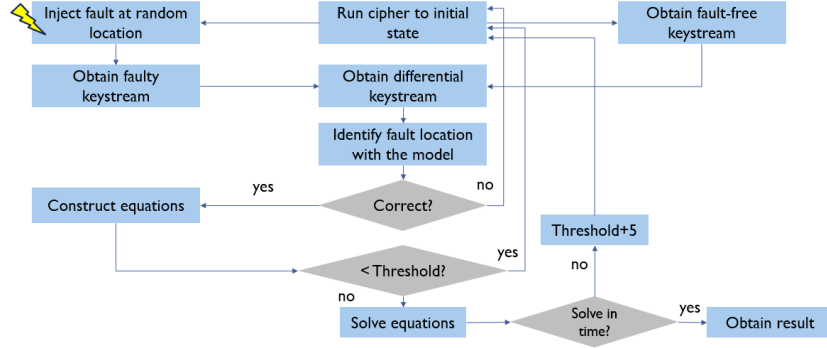


Fig. 1. Threshold-based secret recovery process.

rate, and initial learning rate are selected through empirical experiments. The number of neurons is tuned by adjusting the parameters with a power of two, while the dropout rate is adjusted by increasing or decreasing the rate by 0.05.

3.2 Secret Recovery

Once the MLP models are trained, we proceed to apply the differential fault attack on the targeted ciphers. Figure 1 illustrates the process of secret recovery method. The attack starts by obtaining the fault-free keystream of the cipher. Once the fault-free keystream is obtained, the cipher is reset and run to the initial state. At this stage, a single-bit bit-flipping fault is injected to the initial state of cipher, where the fault location is unknown. We then run the cipher with the faulty initial state and obtain the faulty keystream. The computed differential keystream is then fed into the trained model to identify the fault location. When the fault location is correctly identified, we proceed to the next stage of equations construction. In contrast, the cipher is reset and repeat with the previous process when there is an error in fault location identification.

With the correct fault location, we are able to generate the corresponding faulty keystream equations. By XOR-ing the fault-free and faulty keystream equations, the respective differential keystream equations are obtained. A threshold is introduced to determine the minimum number of differential equations for optimizing the number of fault injections. When the number of differential equations is less than the defined threshold, we repeat the fault injection process. Note that the value of threshold varies across ciphers; this will be discussed in Section 4.

If the number of differential equations has exceeded the threshold, we proceed to solve the equations with Gröbner basis provided in SageMath library. Meanwhile, we set a timer of one minute to determine if Gröbner basis can solve the equations system. If Gröbner basis returns the result within the time, we then verify the recovered secret bits. Otherwise, the current computation is terminated, while the threshold is increased by 5 and the process of fault injection

and equations construction are repeated. Increasing the threshold allows more differential equations to be added into the equations system, providing more information to the Gröbner basis for solving the equations system.

4 Implementation and Results of Attacks

The implementation of the attack framework and the results for each cipher is discussed in this section. Note that the attacks on ACORN and MORUS follow the attack assumption defined in Section 3. For the attack on ATOM, there is a minor change in attack assumption, which we will restate in the assumption in the corresponding section. Due to the structure of the cipher, there are different settings in the model architecture and secret recovery process; these are discussed in detail in the following sections.

4.1 Differential Fault Attack on ACORNv3

For the attack on ACORN, the adversary is assumed to have knowledge of 152-bit keystream. Before the attack, the multilayer perceptron (MLP) model is implemented to facilitate the process of fault location identification. In the attack, the adversary obtains the differential keystream and feeds it into the trained model to identify the fault location and generate the corresponding differential equations with respect to the initial state of the cipher. Once a sufficient number of differential equations is collected, those equations are solved to recover the initial state of ACORNv3.

Implementation of MLP Model for Fault Location Identification Before the attack, three datasets, namely training, testing and validation datasets are generated with Algorithm 4 to train the MLP model for fault location identification for ACORNv3. The training and hyperparameters tuning follow the method defined in Section 3.1. The architecture of the neural network is described as follows:

- **Input layer:** The number of neurons in the input layer is 152.
- **Hidden layer:** A total of five hidden layers are used in this architecture. Each layer is a dense layer that applies a fully connected transformation with L_2 weight decay ($\lambda = 1 \times 10^{-8}$) to prevent overfitting, immediately followed by batch normalization to normalize intermediate activations and mitigate internal covariate shift. ELU activation function is used to combine fast convergence with robustness to vanishing gradients and a dropout rate of 0.3 is set after each activation to prevent co-adaptation of neurons and reduce overfitting. The number of neurons for the five hidden layers are 152, 512, 512, 512 and 512.
- **Output layer:** The neurons in the output layer is activated with the softmax activation function. The number of neurons in this layer is 293, that is the state size of ACORN.

Algorithm 5 Differential Fault Attack on ACORNv3

```
Initialize list  $\mathcal{E}, \mathcal{F}$ 
 $solve \leftarrow \text{false}$ ,  $\mathcal{T} \leftarrow 150$ ,  $num\_deg1 \leftarrow 0$ 
 $Z = \{z_0, z_1, \dots, z_{151}\} \leftarrow encACORN(S, P)$ 
Generate fault-free ANF,  $NKS = \{nks_0, \dots, nks_{151}\}$ 
while  $solve$  is false do
  while  $num\_deg1 < \mathcal{T}$  do
    Reset cipher to initial state  $S'$ 
    Inject fault at location  $f$ , where  $s_f = s_f \oplus 1$ 
     $Z' = z'_0, z'_1, \dots, z'_{151} \leftarrow encACORN(S', P)$ 
    for  $i = 0$  to 151 do
       $\Delta z_i^f = z_i \oplus z'_i$ 
    end for
     $f' \leftarrow modelACORN(\Delta Z^f = \{\Delta z_0^f, \dots, \Delta z_{151}^f\})$ 
    if  $f$  not in  $\mathcal{F}$  and  $f = f'$  then
      Append  $f$  to  $\mathcal{F}$ 
    else
      continue
    end if
    Generate faulty ANF,  $FKS = \{fks_0, \dots, fks_{151}\}$ 
    for  $i = 0$  to 151 do
       $dks = nks_i \oplus fks_i \oplus \Delta z_i^f$ 
      if  $dks.deg = 1$  and  $dks$  not in  $\mathcal{T}$  then
        Append  $dks$  to  $\mathcal{E}$ 
         $num\_deg1 = num\_deg1 + 1$ 
      else if  $dks.deg = 2$  and  $dks$  not in  $\mathcal{T}$  then
        Append  $dks$  to  $\mathcal{E}$ 
      end if
    end for
  end while
  Solve  $\mathcal{E}$  with Gröbner basis
  if Gröbner basis can solve in time then
     $solve \leftarrow \text{true}$ 
  else
     $\mathcal{T} = \mathcal{T} + 5$ 
  end if
end while
```

Implementation of Differential Fault Attack on ACORN The detailed attack process of ACORN is defined in Algorithm 5. Initially, two lists $\mathcal{E}$ and $\mathcal{F}$ are defined to store differential equations and identified fault location, respectively. The setting of threshold $\mathcal{T}$ in this attack is to determine the minimum number of linear equations in the equation system. Through our observation on different settings of threshold, we found that $\mathcal{T}$ of 150 provides an optimal results in state recovery attack. Hence, the threshold $\mathcal{T}$ is set to 150. Before diving into the attack, the adversary obtains the 152-bit fault-free keystream,

$Z = \{z_0, z_1, \dots, z_{151}\}$ and generate fault-free Algebraic Normal Form (ANF) with respect to the initial state of the cipher.

In the online phase, a bit-flipping fault is injected at a random location f at initial state, such that s_f is complement. The adversary utilizes the faulty initial state to obtain the faulty keystream, $Z' = \{z'_0, z'_1, \dots, z'_{151}\}$, and compute the differential keystream, $\Delta Z^f = \{\Delta z_0^f, \Delta z_1^f, \dots, \Delta z_{151}^f\}$ by $\Delta z_i^f = z_i \oplus z'_i$, where $i = 0, 1, \dots, 151$. The differential keystream ΔZ^f is fed into the MLP model to identify the fault location. For the correctly identified fault location f that does not exist in the list $\mathcal{F}$, it is appended to the list. If the location already exists in the list, the adversary proceeds to re-inject a fault at a random location. This process ensure that only unique fault locations are obtained.

After the fault location is identified, the faulty ANFs with respect to the initial state of the cipher, $FKS = \{fks_0, fks_1, \dots, fks_{151}\}$ are generated. The adversary can then generate the differential equation, dks by $nks_i \oplus fks_i \oplus \Delta z_i^f$, where $i = \{0, 1, \dots, 151\}$. Each time a unique linear differential equation is obtained, the counter *num_deg1* is increased by 1 and the equation is added into the equations list $\mathcal{E}$. The obtained unique quadratic equation is also added to the same list $\mathcal{E}$. This process is repeated iteratively until *num_deg1* is greater than or equal to the threshold, $\mathcal{T}$. Subsequently, the system of differential equations in $\mathcal{E}$ are solved using Gröbner basis to recover the initial state S .

Results and Discussions of the Attack on ACORNv3 The performance of the trained model is shown in Table 3. With the proposed MLP architecture, the model achieves a high accuracy of 99.9880%, showing the model has successfully learned the fault propagation pattern to identify the fault locations with 152-bit differential keystream. The precision score of 99.9880% shows that the model is robust and effective to identify the actual fault locations with a negligible false positive rate. In term of recall of 99.9880%, it shows that the model is able to capture nearly all actual fault location, reducing the case of fail detection. The F1-score of 99.9880% demonstrated balanced precision-recall across all target locations, validating the architectural efficacy and hyperparameter configuration.

Table 3. Performance of trained model for identifying fault locations on ACORNv3

Accuracy	Precision	Recall	F1-score
0.999880	0.999880	0.999880	0.999880

On the other hand, the performance of the trained model is benchmarked with the existing work of Set-based fault location identification by Zhang et al. [29]. Besides, a signature-based method is also implemented with the generated dataset, in which the training dataset is used for generating fault signatures, while the testing data for evaluating the performance. Since the differential keystream used by the model is longer than the 99-bit keystream used by Zhang

et al. [29], the benchmarking is done based on the keystream extension method in their work. Considering the case of keystream extension method of 149-bit keystream, the trained model achieves an accuracy that is 0.0106% higher than the Set-based method. For the traditional signature-based method, it possesses the accuracy of 99.9747%, which is 0.0133% lower than the accuracy of our proposed model. Hence, we can conclude that the proposed MLP architecture possesses a high and robust ability to identify the fault location with a higher accuracy.

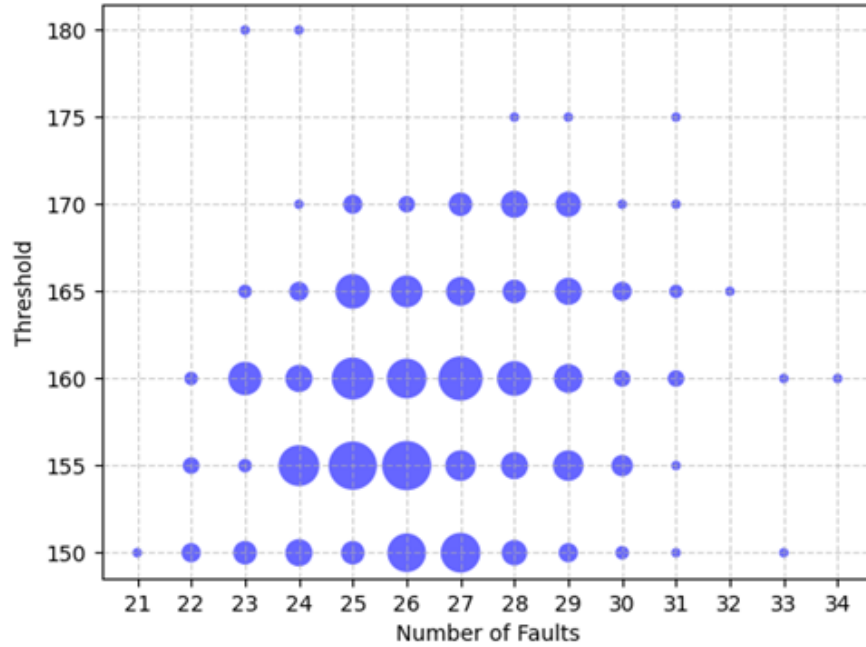


Fig. 2. Number of faults and threshold used in the experiments to recover the initial state of ACORNv3.

In terms of secret recovery, a total of 400 experiments have been conducted to analyse the performance of the proposed method in recovering the initial state of ACORNv3. Excluding eight experiments that possess the case of error identification of fault location, the remaining experiments shows that the 293-bit initial state of ACORNv3 can be fully recovered. The attack has successfully been applied with a high probability of 0.98. Note that the experiment is stopped once there is an error identification. However, even in the case of error identification, the model is still able to identify majority of fault locations, and is expected to recover the majority of the internal state bits.

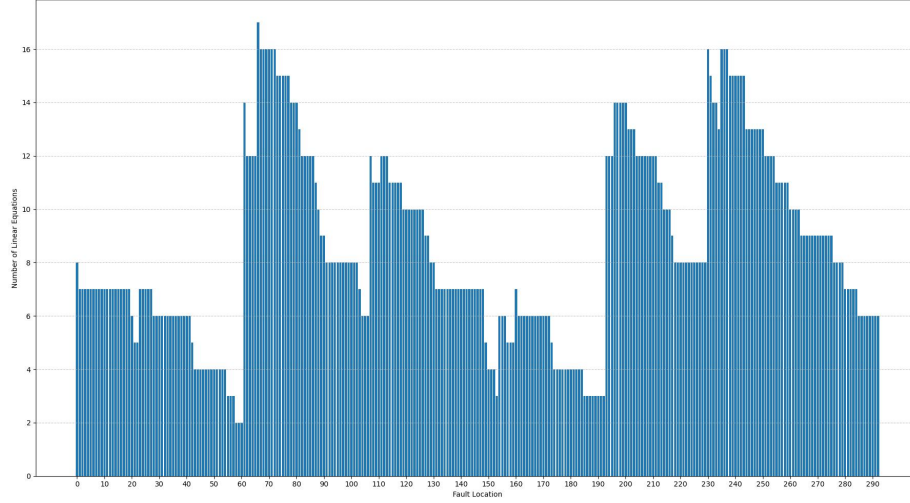


Fig. 3. Number of linear differential equations produce by each fault location in ACORNv3.

Figure 2 shows the thresholds and the number of faults used in the experiments to recover the initial state of ACORNv3. Experimental results demonstrate that the number of faults required to recover the initial state of ACORN ranges between 21 to 34. On average, the number of faults required to recover the initial state of ACORN is 27. In Figure 2, the larger the circle, the more experiments utilized the same number of faults and threshold. The figure shows that the same number of faults may result in a varied range of threshold, as different fault locations can generate different number of linear equations.

Figure 3 shows the number of linear differential equations produced by each fault location in ACORNv3. It is observed that some locations, such as 66 to 77, 230, 231 and 235 to 243 can generate 15 or above linear equations, which are the ideal fault locations. If the fault is injected at these locations, the number of faults required in the attack can be reduced. However, the number of fault injections is also affected by the complexity of the retrieved equations. If the system of equations is complicated for Gröbner basis to solve in time, more fault injections are required.

For the work done by Siddhanti et al. [22], although only a reduced number (9) of faults are required to recover the state, the attacker must know the values of 20 state bits, which increases the complexity of the attack by 2^{20} . For the work done by Zhang et al. [29], the authors focus on recovering the initial state of ACORNv3 by solving the system of linear equations. They required $26 < n < 43$ faults to recover the initial state, using guess-and-determine method, as the number of linear equations is less than the number of state variables. Since their attack requires to guess some state bits, the attack complexity is $c \cdot 2^{146.5-3.52n}$, where c is the time complexity of equation solving and n is the number of

faults. In contrast, our proposed method includes quadratic equations in the state recovery process, achieving full state recovery without adopting the guess-and-determine method. The proposed method successfully recovers the full 293-bit state of ACORNv3 with 21 to 34 faults, that is 19.04% lower than the result of Zhang et al. [29], considering the case of highest number of faults used in the attack. The time complexity of our attack is negligible and significantly lesser than the two existing works.

4.2 Differential Fault Attack on MORUSv2

For the attack on MORUS, the adversary has the knowledge of three plaintext blocks $P = \{P_0, P_1, P_2\}$, i.e., a total of 384-bit plaintext. We implement the MLP model to facilitate the process of fault location identification. During the attack, the adversary obtains the differential keystream and feeds it into the trained model to identify the fault location and generate the corresponding differential equations with respect to the initial state. The same process is repeated until sufficient number of differential equations are collected. The initial state of MORUS are recovered by solving the equations. In the following sections, the initial state is denoted as $S = \{s_0, s_1, \dots, s_{639}\}$.

Implementation of MLP Model for Fault Location Identification With the datasets generated with Algorithm 4, the MLP model for fault location identification for MORUS is implemented. We train the model and tune the hyperparameters with the method defined in Section 3.1. The model architecture is described as follow:

- **Input layer:** This input layer consists of 384 neurons, that is identical to the number of required differential keystream bits.
- **Hidden layer:** A total of three hidden layers are used in this architecture. Each layer is a dense layer followed by batch normalization to mitigate internal covariate shift and increase training stability. For the first two hidden layers, the ReLU activation function and the dropout rate of 0.2 are applied after the batch normalization to reduce overfitting. For the last hidden layer, it only utilizes the ReLU activation after batch normalization. There number of neurons for the first, second, third hidden layers are all 512.
- **Output layer:** The number of neurons in this layer is 640 (number of possible fault locations in MORUS). We utilize softmax as the activation function in this layer.

Implementation of Differential Fault Attack on MORUS The process of differential fault attack on MORUS is described in Algorithm 6. The initial settings of the attack is identical to the attack done on ACORN, except of the setting of the threshold $\mathcal{T}$. The threshold $\mathcal{T}$ is obtained through experimental observation, as in the case of ACORNv3, and set to 896 for MORUS. Unlike ACORNv3, for the application of the attack to MORUS, we include

all equations into the equation system, regardless the degree of equations. Before diving into the attack, the adversary first runs the cipher with fault-free initial state $S = \{s_0, s_1, \dots, s_{639}\}$ and obtains the 384-bit fault-free keystream, $Z = \{z_0, z_1, \dots, z_{383}\}$. Due the constraint of SageMath, we can only generate the ANF for the first 256 iterations of the encryption phase; therefore we construct 256 fault-free keystream equations $NKS = \{nks_0, nks_1, \dots, nks_{255}\}$ $NKS = \{nks_0, nks_1, \dots, nks_{255}\}$.

Algorithm 6 Differential Fault Attack on MORUSv2

```

Initialize list  $\mathcal{E}, \mathcal{F}$ 
 $solve \leftarrow \text{false}$ ,  $\mathcal{T} \leftarrow 896$ ,  $num\_eq \leftarrow 0$ 
 $Z = \{z_0, z_1, \dots, z_{383}\} \leftarrow encMORUS(S, P)$ 
Generate fault-free ANF,  $NKS = \{nks_0, \dots, nks_{255}\}$ 
for  $i = 0$  to 255 do
    Append  $nks_i \oplus z_i$  to  $\mathcal{E}$ 
end for
while  $solve$  is false do
    while  $num\_eq < \mathcal{T}$  do
        Reset cipher to initial state  $S'$ 
        Inject fault at location  $f$ , where  $s_f = s_f \oplus 1$ 
         $Z' = \{z'_0, z'_1, \dots, z'_{383}\} \leftarrow encMORUS(S', P)$ 
        for  $i = 0$  to 383 do
             $\Delta z_i^f = z_i \oplus z'_i$ 
        end for
         $f' \leftarrow modelMORUS(\Delta Z^f = \{\Delta z_0^f, \dots, \Delta z_{383}^f\})$ 
        if  $f$  not in  $\mathcal{F}$  and  $f = f'$  then
            Append  $f$  to  $\mathcal{F}$ 
        else
            continue
        end if
        Generate faulty ANF,  $FKS = \{fks_0, \dots, fks_{255}\}$ 
        for  $i = 0$  to 255 do
             $dks = nks_i \oplus fks_i \oplus \Delta z_i^f$ 
            if  $dks.deg \geq 1$  and  $dks$  not in  $\mathcal{T}$  then
                Append  $dks$  to  $\mathcal{E}$ 
                 $num\_eq = num\_eq + 1$ 
            end if
        end for
    end while
    Solve  $\mathcal{E}$  with Gröbner basis
    if Gröbner basis can solve in time then
         $solve \leftarrow \text{true}$ 
    else
         $\mathcal{T} = \mathcal{T} + 5$ 
    end if
end while

```

In the attack, it follows the same procedure as in the attack on ACORNv3, where the adversary reset the cipher to the initial state, $S' = \{s_0, s_1, \dots, s_{639}\}$ and injects a bit-flipping fault at a random location f , such that s_f is complement. The faulty initial state is then used to run the cipher and output the faulty keystream, $Z' = z'_0, z'_1, \dots, z'_{383}$. With the differential keystream, $\Delta Z^f = \Delta z_0^f, \Delta z_1^f, \dots, \Delta z_{383}^f$, it is fed into the MLP model to identify the fault location. Once the fault location is identified, the ANFs of the first 256-bit faulty keystream with respect to the initial state of the cipher are generated. Whenever a differential equation is retrieved, it is added into the equations system $\mathcal{E}$ regardless of the degree of the equation. Whenever num_eq is less than $\mathcal{T}$, the same process is repeated until the number of equations in the equation system is equal to or exceed the threshold of 896.

When sufficient number of differential equations is collected and stored in $\mathcal{E}$, Gröbner basis is applied for solving the equations system to recover the initial state. This process follows the similar procedure as done in the case of ACORN. Note that in this attack, we also include the first 256 bits original keystream equations into the equation system $\mathcal{E}$ to facilitate the equation solving process, due to the larger state size of MORUS.

Results and Discussions on the Attack on MORUSv2 The performance of the trained MLP model in fault location identification is described in Table 4. The results in Table 4 show that the trained model achieves a high accuracy of 99.9231%, i.e., the model is able to capture most of the pattern of fault propagation. The model achieves the precision score of 99.9235%, showing the model has a high ability to classify the actual fault location with low probability of false positives identification. The recall score of 99.9231% is slightly lower than the precision score by 0.0004%, indicating that the probability of misidentification is slightly higher than the probability of false positives identification. However, the score of 99.9231% is still considered as a good score, as the model can identify most of the actual fault locations among all actual positives case. The F1 score of 99.9231% shows a balanced trade-off between the precision and recall, indicating that the trained model has high reliability in identifying the fault locations.

Table 4. Performance of trained model for identifying fault locations on MORUSv2

Accuracy	Precision	Recall	F1-score
0.999231	0.999235	0.999231	0.999231

To provide a comprehensive view, the traditional signature-based method is also implemented to benchmark the performance of the trained model. For the signature-based method, the same training and testing datasets used for implementing the model are used to generate the fault signature and performing evaluation, respectively. The accuracy of fault location identification for the MLP model is 99.9396%, while the signature-based method is 95.2606%. The result

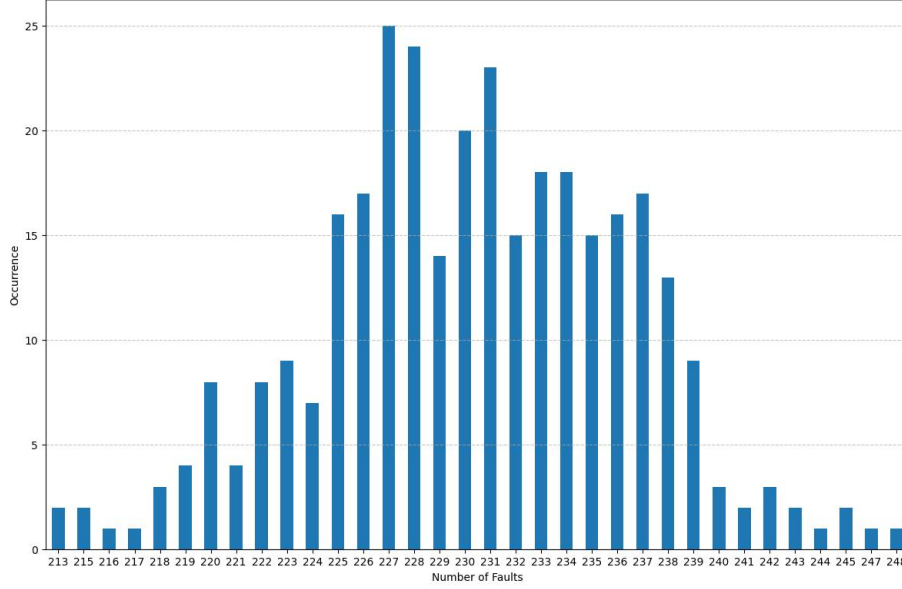


Fig. 4. Frequency of each number of faults used in the successful attacks.

shows that the proposed MLP model outperforms the traditional signature-based method in fault location identification of MORUS. The state update function of MORUS utilizes AND, XOR, and rotation operations to create diffusion and update the state, affecting the propagation of faults within the state. In such case, the MLP model possesses a stronger capability to learn the complex fault propagation pattern, providing a high accuracy in identification. In contrast, the performance of signature-based method that relies on the predefined fault signature to identify the fault location may not be significant, as the propagation of fault in certain locations is non-deterministic due to the diffusion property, causing the error identification.

We have conducted total of 400 experiments to analyze the performance of the proposed state recovery attack on MORUSv2. From the result, 76 out of 400 experiments encounter the case of error identification of fault location. Although the trained model possesses a high accuracy in fault location identification, the large state size of MORUS requires a larger number of faults (213 to 248 faults) to recover the initial state. The increase in number of faults has degraded the performance of model, as the higher number of fault injections has also increased the probability of misidentification. Note that in these 76 erroneous cases, majority of the fault locations can still be identified by the model, but the experiment is set to not proceed to the equation solving phase when there is an error identification. However, it is expected that solving the equation system with error

identification can recover majority of the state bits, as the number of correct identifications is significantly larger than the error identification cases.

In the remaining 324 cases, the results show that the full initial state of MORUSv2 can be recovered. Among the 324 successful attacks, 86 attacks directly recover the state by solving equations, while 238 attacks require the adversary to guess a few state bits in order to achieve full state recovery. For these 238 attacks, the recovered state bits can be further categorized into three groups: the direct recovered bits, indirect recovered bits and guessing bits. The direct recovered bits refer to the state bits that can be directly recovered by solving the equations. The guessing bits are the state bits that cannot be recovered by solving the equations, and the adversary to guess the value to recover the state. The indirect recovered bits are the state bits that can be recovered from the equations that involved one or more guessing bits. To recover the indirect recovered bits correctly, the adversary needs to guess the value of the guessing bits correctly. In these attack, the number of direct recovered bits is ranging from 600 to 639, while the number of indirect recovered bits is ranging from 0 to 35 and the number of guessing bits is ranging from 1 to 6. Considering the highest number of guessing bits of 6, the adversary only needs to try for $2^6 = 64$ possible combinations at most. With the correct combination, the indirect recovered bits can also be recovered, and thus recover the full initial state of the cipher. The average number of faults required to recover the initial state is 231 faults. Figure 4 shows the number of faults and the number of successful attacks (occurrence) utilized that exact number of faults to recover the initial state of MORUS.

The attack proposed by Wong et al. [24] can recover only two register's contents under the assumption of multi-byte random fault with precise control over fault location. In contrast, our proposed attack utilized a more relaxed assumption of single-bit bit-flipping fault with no-control over fault location. Compared to Wong et al., our proposed attack require a weaker fault model and can recover the full initial state of MORUSv2. Although Wong et al. [24] performed the theoretical analysis, they did not implement the attack on the cipher. Hence, the proposed method is considered as the first implementation of differential fault attack against MORUS. The time complexity of the attack is mainly the time complexity of Gröbner basis solving the equation and guessing the values of h guessing bits, where $h \in \{1, 2, 3, 4, 5, 6\}$. Since the time complexity of Gröbner basis solving the equations is negligible, the overall complexity of the attack is at most 2^h .

4.3 Differential Fault Attack on ATOM

In the attack on ATOM, we assumed that the adversary knows 56 bits plaintext $P = \{p_0, p_1, \dots, p_{55}\}$. Due to the design of the cipher, the attack assumption made for this attack is slightly stronger than the attacks in previous sections. Since the *cnt* in the keystream generation phase for the key filter depends on the state of LFSR, the adversary needs to possess the information of the LFSR state to determine which key bits are involved in the operation. Hence, it is assumed

that the adversary knows the first 60-bit content in the LFSR. The adversary does not need to guess the value of the last 9-bit of the LFSR, as they are always with the value 1 at the beginning of encryption phase. The fault is assumed to be injected at the initial state of NFSR.

Implementation of MLP Model for Fault Location Identification The implementation of the MLP model for ATOM is focused on identifying the fault location in the NFSR. Thus, the generated datasets only include samples of faults injected in the NFSR. The training and hyperparameters tuning follow the same method as in previous sections. The architecture of the developed MLP model is described as follows:

- **Input layer:** The number of neurons in the input layer is 56.
- **Hidden layer:** A total of three hidden layers are used. Each layer is a dense layer followed by batch normalization to mitigate the issue of internal covariate shift. For the first two hidden layers, the ELU activation function and the dropout rate of 0.2 are applied after the batch normalization to reduce overfitting. The last hidden layer only utilizes the ELU activation after batch normalization. There number of neurons for the first, second, third hidden layers are 128, 256, 128, respectively.
- **Output layer:** There is a total of 90 neurons in this layer (number of possible fault locations in the NFSR of ATOM), using softmax as the activation function.

Implementation of Differential Fault Attack on ATOM We describe the implementation of DFA on ATOM in Algorithm 7. In this attack, the threshold $\mathcal{T}$ is set to 234. As the minimum of 90 differential equations is required to recover the NFSR, with 128-bit secret key variables, and additional 16 original equations are included in the equation system, the threshold is set to 234. Note that only partial of the secret key bits can be recovered, as SageMath can only generate the first 16-bit keystream equations, which are not sufficient for recovering all secret key bits. The adversary first includes the original keystream equations into $\mathcal{E}$ by XOR-ing nks_i with the corresponding keystream bit z_i , where $i = \{0, 1, \dots, 15\}$.

During the attack, a bit-flipping fault is injected at a random location f in the initial state of NFSR B , such that b_f is complement. Once the differential keystream, $\Delta Z^f = \Delta z_0^f, \Delta z_1^f, \dots, \Delta z_{55}^f$ is obtained, it is inputted into the model to identify the fault location. Whenever the fault location is correctly identified, the differential equations dks are retrieved by $nks_i \oplus fks_i \oplus \Delta z_i^f$, where $i = \{0, 1, \dots, 15\}$, and added to $\mathcal{E}$, without considering the degree of the equation. The same process of differential equation retrieving is repeated until the number of equations in the equation system is equal to or exceed the threshold of 234.

When number of differential equations stored in $\mathcal{E}$ exceeds the set threshold, the equations system is solved using Gröbner basis to recover the initial NFSR state and the secret key. The remaining process is identical to the process done in the case of ACORNv3 and MORUSv2 in equation solving. Whenever the

solution is not returned in time, the threshold is increased by 5 to include more equation in the equation system for enhancing the equation solving process by Gröbner basis.

Algorithm 7 Differential Fault Attack on ATOM

```

Initialize list  $\mathcal{E}, \mathcal{F}$ 
 $solve \leftarrow \text{false}$ ,  $\mathcal{T} \leftarrow 234$ ,  $num\_eq \leftarrow 0$ 
 $Z = \{z_0, z_1, \dots, z_{55}\} \leftarrow encATOM(S, P)$ 
Generate fault-free ANF,  $NKS = \{nks_0, \dots, nks_{15}\}$ 
for  $i = 0$  to 15 do
    Add  $nks_i \oplus z_i$  to  $\mathcal{E}$ 
     $num\_eq = num\_eq + 1$ 
end for
while  $solve$  is false do
    while  $num\_eq < \mathcal{T}$  do
        Reset cipher to initial state  $B', L$ 
        Inject fault at location  $f$ , where  $b_f = b_f \oplus 1$ 
         $Z' = \{z'_0, z'_1, \dots, z'_{55}\} \leftarrow encATOM(B', L)$ 
        for  $i = 0$  to 55 do
             $\Delta z_i^f = z_i \oplus z'_i$ 
        end for
         $f' \leftarrow modelATOM(\Delta Z^f = \{\Delta z_0^f, \dots, \Delta z_{55}^f\})$ 
        if  $f$  not in  $\mathcal{F}$  and  $f = f'$  then
            Append  $f$  to  $\mathcal{F}$ 
        else
            continue
        end if
        Generate faulty ANF,  $FKS = \{fks_0, \dots, fks_{15}\}$ 
        for  $i = 0$  to 15 do
             $dks = nks_i \oplus fks_i \oplus \Delta z_i^f$ 
            if  $dks.deg \geq 1$  and  $dks$  not in  $\mathcal{T}$  then
                Append  $dks$  to  $\mathcal{E}$ 
                 $num\_eq = num\_eq + 1$ 
            end if
        end for
    end while
    Solve  $\mathcal{E}$  with Gröbner basis
    if Gröbner basis can solve in time then
         $solve \leftarrow \text{true}$ 
    else
         $\mathcal{T} = \mathcal{T} + 5$ 
    end if
end while

```

Result and Discussion on the Attack on ATOM Table 5 shows the performance of the MLP model for ATOM. It shows the performance of the trained model achieves an accuracy of 82.3568%. Compared to the trained models for

ACORNv3 and MORUSv2, this model has a lower accuracy on ATOM. Although the model can achieve an accuracy more than 80%, the model possesses high probability to encounter the error identification. However, it is also affected by the design of the cipher. As ATOM utilizes double key filter, and the setting of *cnt* in the encryption phase is based on the current state of LFSR, the pattern of the fault propagation will be highly non-deterministic, that cause the model faces difficulties in learning the fault propagation pattern. The model achieves the precision score of 82.1186%, indicating that there is a chance that the model will encounter false positives identification. The recall score of 82.3568% is slightly higher than the precision score, showing that the probability of misidentification is slightly lower than the probability of false positives identification. The F1 score of 81.6163% shows a balanced trade-off between the precision and recall.

Table 5. Performance of trained model for identifying fault locations on ATOM

Accuracy	Precision	Recall	F1-score
0.823568	0.821186	0.823568	0.816163

On the other hand, the traditional signature-based method is also implemented to benchmark the performance of the trained model. The same training and testing datasets are used to generate the fault signature and performing evaluation for the signature-based method. The accuracy of the MLP model is 82.3568%, while the accuracy for the signature-based method is 58.8976%. The results show that the traditional signature-based method has a poor performance in fault location identification. Overall, the MLP model has a significantly improved performance than the signature-based method to learn the complex fault propagation pattern, providing a higher accuracy than signature-based method in fault location identification. Although the performance of this model is not as significant as the previous trained models for ACORN and MORUS, it still provides an insight that the deep learning model possesses high ability than traditional method to learn the highly non-deterministic fault propagation pattern.

A total of 400 experiments have been conducted to analyze the performance of the proposed state recovery and key recovery attack on ATOM. Unfortunately, all experiments have encountered the case of wrong identification. Due to the setting of the experiments, we do not proceed to equation solving if there is any error identification occurred. This has shown that the utilization of double key filter provides a high quality of security margin, as the adversary would not be able to collect an all correctly identified fault location to perform the state recovery and key recovery process. If the adversary utilizes the result of wrong identification to generate the equation system and solve the system, it is expected that some of the state bits or the key bits will not be recovered correctly. In these experiments, a new fault is injected to the NRSR of the cipher whenever the number of equations in the system does not exceed the threshold of 234. To reach the threshold of 234, it requires on average 46 faults.

Since the attack based on the model of no control over the fault location cannot recover any initial NFSR state and secret key bit, an experiment is conducted based on the fault model of precise control over the fault locations. In this experiment, the utilization of threshold is excluded. Based on the previous experiments, 46 faults are needed to construct the equation system with at least 234 equations. Suppose that the adversary repeatedly reset the cipher and injects each fault in $\mathcal{F}$, where $\mathcal{F}$ consists of 46 faults that are defined as follows:

$$\begin{aligned} \mathcal{F} = \{ & b_3, b_7, b_9, b_{10}, b_{11}, b_{16}, b_{17}, b_{20}, b_{21}, b_{22}, b_{24}, b_{25}, b_{30}, \\ & b_{31}, b_{34}, b_{35}, b_{36}, b_{37}, b_{40}, b_{41}, b_{42}, b_{44}, b_{46}, b_{48}, b_{49}, \\ & b_{52}, b_{53}, b_{54}, b_{56}, b_{58}, b_{62}, b_{63}, b_{65}, b_{67}, b_{69}, b_{72}, b_{73}, \\ & b_{75}, b_{76}, b_{79}, b_{82}, b_{83}, b_{84}, b_{85}, b_{86}, b_{89} \}. \end{aligned} \quad (10)$$

Note that the fault location in $\mathcal{F}$ are chosen randomly. By solving the differential equations generated from the faults above, including the original keystream equation, most of the state bits in the NFSR are recovered. There are 74 out of 90 state bits can be recovered. Note that some of the state bits are recovered with wrong values, which may be affected by the insufficient length of keystream, causing Gröbner basis to retrieve an error result. All the key bits involved in the equations need to be guessed to solve the equation systems.

5 Conclusion

This work focuses on investigating the feasibility of deep learning assisted differential fault attack against ACORNv3, MORUSv2 and ATOM with the fault model of single-bit bit flipping fault with no control over the fault location. We have implemented three MLP models for identifying the fault location on ACORNv3, MORUSv2 and ATOM, where the accuracies of the trained models have improved the traditional signature-based method. Besides, we propose a threshold-based strategy to optimize the number of fault injections. The results show that we can recover the initial state of ACORN with the complexity of negligible and achieve full state recovery on MORUS, that are better than the results of existing works. On the other hand, we have implemented the first DFA on ATOM and managed to recover majority of state bits in NFSR under the fault model of precise control over the fault location. It shows that the design of double key filter has improved the security margin of the cipher. Future works may examine the feasibility of deep learning assisted fault attack under the more relaxed fault attack model of random fault with no control over the fault location. Besides, one may consider to remove the timer used in equation solving, to examine the number of faults needed to recover the state. Furthermore, one can proceed to solve the equation that encounter error in fault location identification, as it is expected that some of the state bits can be recovered correctly.

References

1. Baksi, A., Sarkar, S., Siddhanti, A., Anand, R., Chattopadhyay, A.: Differential fault location identification by machine learning. *CAAI Transactions on Intelligence Technology* **6**(1), 17–24 (2021). <https://doi.org/10.1049/cit2.12027>
2. Banik, S., Caforio, A., Isobe, T., Liu, F., Meier, W., Sakamoto, K., Sarkar, S.: Atom: A stream cipher with double key filter. *IACR Transactions on Symmetric Cryptology* **2021**(1), 5–36 (Mar 2021). <https://doi.org/10.46586/tosc.v2021.i1.5-36>
3. Biham, E., Shamir, A.: Differential fault analysis of secret key cryptosystems. In: Kaliski, B.S. (ed.) *Advances in Cryptology — CRYPTO '97*. pp. 513–525. Springer, Berlin, Heidelberg (1997). <https://doi.org/10.1007/BFb0052259>
4. Boneh, D., DeMillo, R.A., Lipton, R.J.: On the importance of checking cryptographic protocols for faults. In: Fumy, W. (ed.) *Advances in Cryptology — EUROCRYPT '97*. pp. 37–51. Springer, Berlin, Heidelberg (1997). https://doi.org/10.1007/3-540-69053-0_4
5. Cheng, Y., Ou, C., Zhang, F., Zheng, S., Xu, S., Long, J.: DLFA: Deep learning based fault analysis against block ciphers. *Cryptology ePrint Archive*, Paper 2023/021 (2023), <https://eprint.iacr.org/2023/021>
6. Dalai, D.K., Roy, D.: A state recovery attack on ACORN-v1 and ACORN-v2. In: Yan, Z., Molva, R., Mazurczyk, W., Kantola, R. (eds.) *Network and System Security*. pp. 332–345. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-64701-2_24
7. Gohr, A.: Improving attacks on round-reduced Speck32/64 using deep learning. In: *Advances in Cryptology – CRYPTO 2019: 39th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part II. p. 150–179. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26951-7_6
8. Hoch, J.J., Shamir, A.: Fault analysis of stream ciphers. In: Joye, M., Quisquater, J.J. (eds.) *Cryptographic Hardware and Embedded Systems - CHES 2004*. pp. 240–253. Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-28632-5_18
9. Hou, Z., Ren, J., Chen, S.: Improve neural distinguishers of SIMON and SPECK. *Security and Communication Networks* **2021**(1), 9288229 (2021). <https://doi.org/10.1155/2021/9288229>
10. Kim, H., Lim, S., Kang, Y., Kim, W., Kim, D., Yoon, S., Seo, H.: Deep-learning-based cryptanalysis of lightweight block ciphers revisited. *Entropy* **25**(7) (2023). <https://doi.org/10.3390/e25070986>
11. Ma, Z., Tian, T., Qi, W.: Differential fault attack on the stream cipher LIZARD. *Chinese Journal of Electronics* **30**(3), 534–541 (2021). <https://doi.org/10.1049/cje.2021.04.007>
12. Maitra, S., Sarkar, S., Baksi, A., Dey, P.: Key recovery from state information of Sprout: application to cryptanalysis and fault attack. *Cryptology ePrint Archive*, Paper 2015/236 (2015), <https://eprint.iacr.org/2015/236>
13. Maitra, S., Siddhanti, A., Sarkar, S.: A differential fault attack on Plantlet. *IEEE Transactions on Computers* **66**(10), 1804–1808 (2017). <https://doi.org/10.1109/TC.2017.2700469>
14. Mondal, S.K., Dey, P., Roy, H.S., Adhikari, A., Maitra, S.: Improved fault analysis on Subterranean 2.0. *IEEE Transactions on Computers* **73**(6), 1631–1639 (2024). <https://doi.org/10.1109/TC.2024.3371784>

15. Orumiehchiha, M.A., Rostami, S., Shakour, E., Pieprzyk, J.: A differential fault attack on the WG family of stream ciphers. *Journal of Cryptographic Engineering* **10**(2), 189–195 (Jun 2020). <https://doi.org/10.1007/s13389-020-00222-x>
16. Prajasantosa, S.R., Salam, I.: Differential fault analysis of TinyJAMBU. In: Boyd, C., Safavi-Naini, R., Simpson, L. (eds.) *Information Security in a Connected World: Celebrating the Life and Work of Ed Dawson*. pp. 68–88. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-83490-5_4
17. Radheshwar, R., Roy, D.: Differential fault attack on ChaosForge. In: 2025 6th International Conference on Recent Advances in Information Technology (RAIT). pp. 1–6 (2025). <https://doi.org/10.1109/RAIT65068.2025.11089261>
18. Rostami, S., Orumiehchiha, M.A., Shakour, E., Alizadeh, S.: Fault attack on enocoro stream cipher family. *Journal of Cryptographic Engineering* **15**, 3 (Jan 2025). <https://doi.org/10.1007/s13389-024-00367-z>
19. Saha, S., Alam, M., Bag, A., Mukhopadhyay, D., Dasgupta, P.: Learn from your faults: Leakage assessment in fault attacks using deep learning. *Journal of Cryptology* **36**(3), 19 (May 2023). <https://doi.org/10.1007/s00145-023-09462-6>
20. Salam, I., Ooi, T.H., Xue, L., Yau, W.C., Pieprzyk, J., Phan, R.C.W.: Random differential fault attacks on the lightweight authenticated encryption stream cipher Grain-128AEAD. *IEEE Access* **9**, 72568–72586 (2021). <https://doi.org/10.1109/ACCESS.2021.3078845>
21. Salam, I., Yau, W.C., Phan, R.C.W., Pieprzyk, J.: Differential fault attacks on the lightweight authenticated encryption algorithm CLX-128. *Journal of Cryptographic Engineering* **13**, 265–281 (Sep 2023). <https://doi.org/10.1007/s13389-023-00326-0>
22. Siddhanti, A., Sarkar, S., Maitra, S., Chattopadhyay, A.: Differential fault attack on Grainv1, ACORNv3 and Lizard. In: Ali, S.S., Danger, J.L., Eisenbarth, T. (eds.) *Security, Privacy, and Applied Cryptography Engineering – SPACE 2017*. pp. 247–263. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-71501-8_14
23. Silva, C., Tenório, N., Bernardino, J.: Lightweight encryption algorithms for IoT. *Computers* **14**(12) (2025). <https://doi.org/10.3390/computers14120505>
24. Wong, K.K.H., Bartlett, H., Simpson, L., Dawson, E.: Differential random fault attacks on certain CAESAR stream ciphers (supplementary material). *Cryptology ePrint Archive*, Paper 2020/022 (2020), <https://eprint.iacr.org/2020/022>
25. Wu, H.: Acorn: A lightweight authenticated cipher (v3) (2016), <https://competitions.cr.yp.to/round3/acornv3.pdf>
26. Wu, H., Huang, T.: The authenticated cipher MORUS (v2) (2016), <https://competitions.cr.yp.to/round3/morusv2.pdf>
27. Zahednejad, B., Lyu, L.: An improved integral distinguisher scheme based on neural networks. *International Journal of Intelligent Systems* **37**(10), 7584–7613 (2022). <https://doi.org/10.1002/int.22895>
28. Zhang, X., Feng, X., Lin, D.: Fault attack on the authenticated cipher ACORNv2. *Security and Communication Networks* **2017**(1), 3834685 (2017). <https://doi.org/10.1155/2017/3834685>
29. Zhang, X., Feng, X., Lin, D.: Fault attack on ACORN v3. *The Computer Journal* **61**(8), 1166–1179 (05 2018). <https://doi.org/10.1093/comjnl/bxy044>